Deloitte.

Automação de Testes

Quality Assurance

Por que utilizar a automação em testes?

- Evitar repetição
- Aumentar a velocidade
- Melhorar a confiabilidade
- Possibilita maior cobertura
- Detecta bugs mais cedo (CI/CD)
- Reduz custos

Playwright

- Framework de testes de ponta a ponta para aplicações web modernas
- Surgiu em 2020
- Criado pela Microsoft
- Testes visuais, testes de componentes, testes de integração, testes funcionais, testes E2E, testes de API


Playwright


cypress


Se
Selenium


appium


TRICENTIS
Tosca


POSTMAN


tR
testRigor

Critério	Playwright	Cypress	Selenium
Arquitetura	Direto no navegador	Dentro do navegador	Via WebDriver
Estabilidade	Alta (auto-wait)	Alta	Média (manual)
Navegadores	Chrome, Edge, Firefox, Safari	Chrome, Edge, Firefox	Chrome, Edge, Firefox, Safari, IE
CI/CD	Integração fácil	Integração boa	Mais configuração
Linguagens	JS, TS, Python, Java, C#	JS, TS	Java, Python, C#, JS, Ruby

Estrutura do arquivo

1. imports
2. test.beforeAll
3. test.describe
4. test.beforeEach
5. test
6. test.afterEach
7. test.afterAll

Localizando elementos pela estrutura HTML

Tipo	Exemplo
Por nome da tag	await page.locator(‘input’).click() await page.locator(‘input’).first().click()
Por ID	Await page.locator(‘#login’).click()
Por um valor da classe	page.locator('.shape-rectangle')
Por classe inteira	page.locator(’[class”input-full-width size-medium status-basic shape-rectangle nb-transition”]’)
Por atributo:	page.locator(’[placeholder=”Email”]’)

Localizando elementos pelo signficado

Tipo	Exemplo
Por role:	await page.getByRole('textbox', {name: 'Email'}).first().click() await page.getByRole('button', {name: 'Sign in'}).first().click()
Por label:	await page.getByLabel('Email').first().click()
Por placeholder:	await page.getByPlaceholder('Jane Doe').click()
Por texto:	await page.getByText('Using the Grid').click()
Por id de teste:	await page.getByTestId('SignIn').click()
Por título:	await page.getByTitle('Iot Dashboard').click()

Comandos básicos

- Rodar projeto:

```
npx playwright test
```

- Rodar pasta de arquivos de testes em um navegador específico:

```
npx playwright test --project=chromium
```

- Rodar um arquivo de teste específico:

```
npx playwright test nomedoarquivo.ts
```

- Rodar somente um teste específico:

```
npx playwright test -g “título do teste”
```

Comandos básicos

- Rodar teste visualizando as interações:

```
npx playwright test --headed
```

- Rodar relatório dos testes:

```
npx playwright show-report
```

Setup

1. Instalar Visual Studio Code
2. Instalar Git
3. Instalar Node (Versão LTS)
4. Dentro do VS Code: Instalar extensão - Playwright Test for VSCode
5. Criar e abrir uma pasta a qual deseja realizar o projeto
6. No terminal, rodar npm init playwright@latest
7. Happy hacking!

Exploração

1. Conhecer principais ferramentas
2. Comandos básicos
3. Execução ui: npx playwright test –ui
4. Conhecer debug: npx playwright test –debug
5. Test.only e Test.skip

Codespaces

- O comando abaixo inicializa um novo projeto ou adiciona o Playwright a um projeto existente:
  npm init playwright@latest
- Instalamos as dependências necessárias:
  npx playwright install --with-deps

interface gráfica virtual

sudo apt update

sudo apt install xvfb

Executar

xvfb-run npx playwright test --ui

Playwright

↓

xvfb-run

↓

Cria DISPLAY virtual

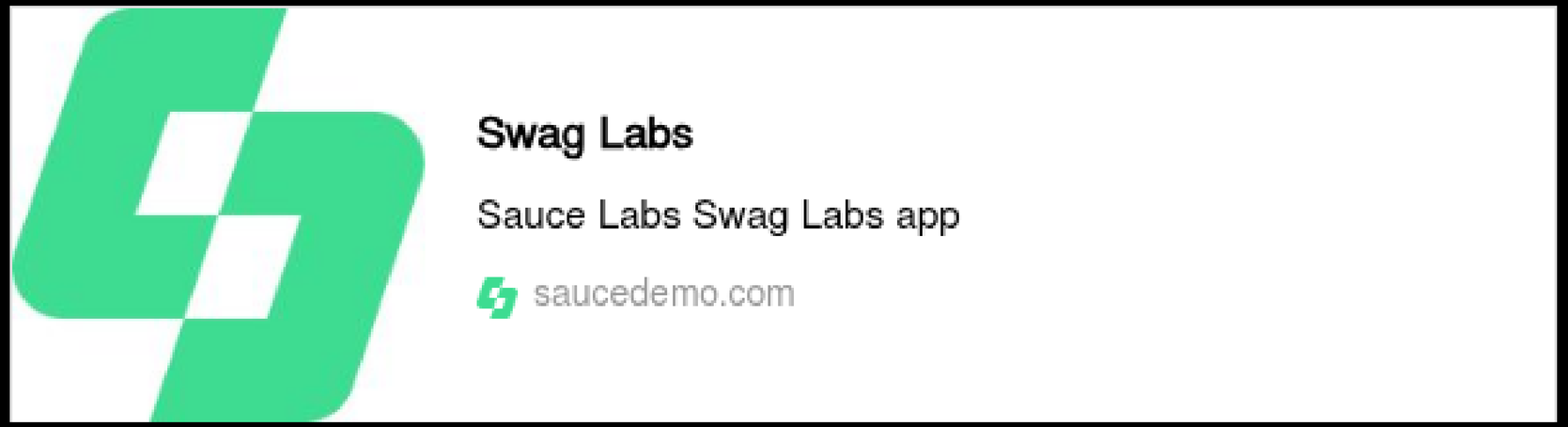
↓

Chromium inicia

↓

Teste executa

Mão na Massa



test: "Os produtos devem ser exibidos com imagem, descrição, detalhes e preço"

```
import { test, expect } from "@playwright/test";


test("Os produtos devem ser exibidos com imagem, descrição, detalhes e preço", async ({page}) => {
```

Caso de teste simples:

```
test("Os produtos devem ser exibidos com imagem, descrição, detalhes e preço", async ({ page }) => {
  // Acessa a página de login
  await page.goto("https://www.saucedemo.com/");
  Ctrl+L for Agent
  // Login executado diretamente, sem chamar funções externas
  await page.locator('[data-test="username"]').fill("standard_user");
  await page.locator('[data-test="password"]').fill("secret_sauce");
  await page.locator('[data-test="login-button"]').click();
```

```
// Verifica se os produtos estão visíveis
  let imagem = await page.locator('.inventory_item_img img').first().getAttribute('src');
  let descricao = await page.locator('.inventory_item_name').first().textContent();    As exp
  let descricaoDetalhada = await page.locator('.inventory_item_desc').first().textContent();
  let preco = await page.locator('.inventory_item_price').first().textContent();    As expres
```

Caso de teste simples:

```
// Exibe as informações dos produtos no console
console.log("Imagem do produto: " + imagem);
console.log("Descrição do produto: " + descricao);
console.log("Descrição detalhada do produto: " + descricaoDetalhada);
console.log("Preço do produto: " + preco);
```

```
// Verifica se as informações dos produtos estão definidas
expect(imagem).toBeDefined();
expect(descricao).toBeDefined();
expect(descricaoDetalhada).toBeDefined();
expect(preco).toBeDefined();
```

Obrigado!